ENGR 852 Motion Estimator

V4 design, verification, and 14 nm implementation record | November 4, 2026

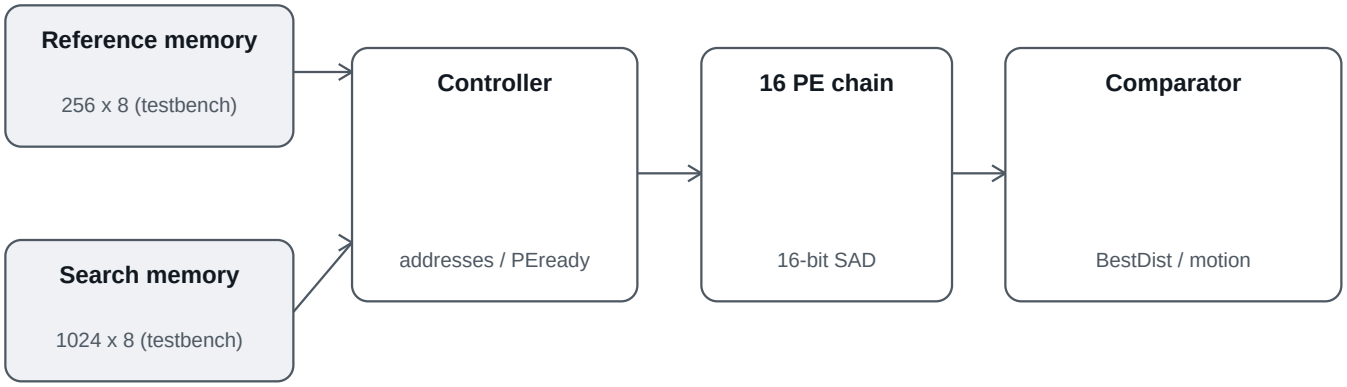
Document scope: actual VCS/Verdi verification, full-precision SAD correction, mapped SAED14 synthesis measurements, physical-flow checkpoint, and outstanding sign-off evidence. Measurements are labeled by their flow stage.

---

1. Design target and memory boundary

The design compares an 8-bit grayscale **16 x 16** reference block against candidate positions in a **31 x 31** search window. A 16-PE systolic datapath performs absolute differences and accumulation while the controller generates search/reference addresses and candidate vectors. The comparator selects the minimum sum of absolute differences (SAD).

External testbench memories are NOT synthesized inside top.



Reference/search data and addresses cross the memory interface

Course setting	Required value
Clock frequency and cycle	520 MHz; 1.923 ns period
Input and output delay	0.500 ns each
Clock uncertainty	0.200 ns
Reference/search memory	Outside synthesizable top; testbench models drive R, S1, S2
SAD precision	16 bits to accommodate 256 x 255 = 65,280 (FF00)

The provided top-level RTL has memory-data inputs R/S1/S2 and address outputs AddressR/AddressS1/AddressS2; only the controller, processing elements and comparator belong in the physical chip.

2. Functional RTL validation

The original source ZIP was preserved separately. Baseline directed and randomized tests were executed with Synopsys VCS W-2024.09-1; Verdi W-2024.09-1 displayed genuine FSDB waveform data. The clock toggles every 0.9615 ns, producing the target period of 1.923 ns at timescale 1ns/100fs.

Stimulus / scenario	Observed result	Interpretation
Directed reference/search fixture	BestDist=00; X=7; Y=8; candidate count 271	Perfect match, signed vector (+7,-8)
Single-pixel +3 perturbation	BestDist=03; X=E; Y=D; candidate count 1542	Nonzero SAD, signed vector (-2,-3)
Upper-corner boundary	BestDist=00; X=7; Y=7; count 4111	Last candidate, signed vector (+7,+7)
PE / PEend / comparator directed	12 checks, 0 failures	Saturation, mux path, strict-less-than behavior
Randomized saturating-PE scoreboard	3 seeds, 24,576 PE comparisons, 0 mismatches	The original 8-bit *saturating* behavior matches its reference

Interpretation: the randomized PE scoreboard establishes agreement with the defined saturating 8-bit behavior; it does **not** prove that 8-bit SAD gives the correct minimum for every image. This motivated the separate full-precision correction.

2.1 Reproduced saturation defect

For the directed all-high-contrast fixture, all 256 reference pixels are 00 and every search pixel is FF. Each candidate has a true SAD of 256 x 255 = 65,280 (hex FF00). Original PEs saturate at 8-bit FF. The original comparator initializes BestDist to FF and updates only when newDist is **strictly smaller**, so no saturated candidate can be accepted.

Output	Original RTL / VCS observation	Corrected variant / VCS observation
BestDist	FF	FF00
motionX / motionY	X / X (unknown)	8 / 8 = signed (-8,-8)
completed	1	1
Comparator newBest	0; no valid winner	1 on first equal-SAD candidate

The 16-bit fix widens the PE accumulators, comparator datapath and best-distance register. The independent V3B hold variant also avoids output changes after completion; its directed run retained the corrected result for 13 further clock cycles. The professor source and teammate implementation were not overwritten.

3. Native waveform evidence

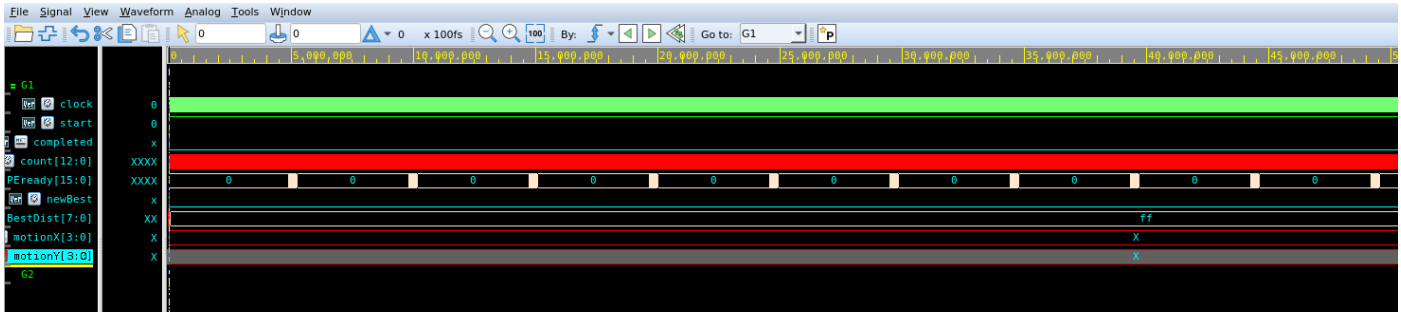


Figure 1. Original 8-bit VCS trace displayed in Verdi nWave. PEready pulses occur but newBest stays low; BestDist remains FF and motionX/Y are unknown.

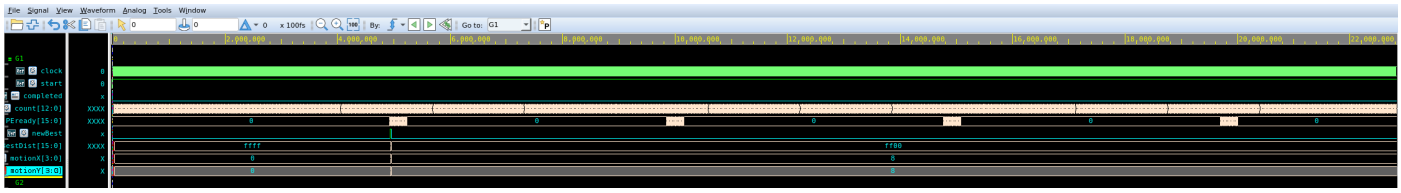


Figure 2. Corrected 16-bit Verdi nWave trace. NewBest updates BestDist to FF00, and the first candidate is encoded as 8,8 (signed -8,-8).

Both images were captured from the nWave application window without including unrelated desktop applications. Cropping was limited to the waveform panels; the original raw evidence is preserved in earlier V3 records.

As in the team controller slide, the signal list includes clock, start, completed, count[12:0], PEready[15:0], newBest, BestDist and motionX/Y. This contribution focuses on the comparator and SAD-precision failure rather than repeating the controller-row demonstration.

4. SAED14 Fusion Compiler synthesis results

The instructor-prescribed local Fusion Compiler setup loaded X-2025.06-SP2a and SAED14 RVT libraries. The example environment and motion-estimator mapped compile completed. The numbers below were read from native Synopsys reports and are **mapped-cell measurements**, not final routed core area.

Report metric	Original 8-bit RTL	Corrected 16-bit V3B
Mapped standard-cell area	881.96 um2	1,193.03 um2
Mapped cell count	1,693	1,905
Combinational cells	1,413	1,489
Sequential cells	280	416
Memory macros / black boxes	0	0
Compile-stage setup slack	First SDC run invalid input-delay collection	+0.06 ns (MET)

Corrected implementation area grows by **311.07 um2 (35.3%)** relative to the original. The 16-bit datapath preserves SAD values up to 65,280 without loss. Timing impact requires further optimization and post-route analysis.

4.1 Timing methodology

• Clock 1.923 ns and uncertainty 0.200 ns are set in the design constraints.

• External input delay and output delay are each 0.500 ns.

• An unsupported expression in the first motion-estimator input-delay command was corrected, and the reloaded timing path confirmed 0.500 ns input delay and 0.200 ns uncertainty.

• The measured +0.06 ns path is a compile-stage max-delay setup report. It is **not** a post-layout PrimeTime setup or hold measurement.

4.2 Power assumptions

The mapped flow can produce preliminary power reports, but a verified numeric final power figure and activity assumptions were not recovered in the evidence available for this report. A legitimate final power value must identify the cell library corner, clock rate, toggle/activity basis, and post-route extracted interconnect model. A nominal estimate is not inserted as a measured result.

5. Cycle-Accurate RTL Verification, Stimulus Analysis & Verdi Waveforms

This section documents the complete behavioral and cycle-accurate verification of both the baseline 8-bit Motion Estimator RTL and the full-precision 16-bit SAD RTL at **520 MHz (1.923 ns clock period, 1ns/100fs timescale, #0.9615 half-period)** using Synopsys VCS (W-2024.09-1) and Verdi nWave (W-2024.09-1). Per course requirements, reference memory (**ROM_R**, 256x8) and search memory (**ROM_S**, 1024x8) are instantiated strictly in the Verilog testbench and excluded from synthesis.

5.1 Why BestDist is Input-Dependent (0000 vs. 0003 vs. FF00) and Summary of RTL Changes

In a 16x16 block-matching motion estimator with 8-bit grayscale pixels, each candidate displacement accumulates 256 absolute pixel differences |R[k] - S[k]| in [0, 255]. Consequently, the true mathematical Sum of Absolute Differences (SAD) spans **0 to 256 x 255 = 65,280 (16'hFF00)**. In the original 8-bit RTL, each PE accumulator saturates any sum >= 255 to **8'hFF**, which equals the comparator's reset sentinel (**BestDist = 8'hFF**). Because the comparator uses strict inequality (**newDist < BestDist**), if all 256 candidates exceed 254 (such as an all-00 reference against an all-FF search window), **newBest** never asserts and **motionX/motionY** remain uninitialized (**X**).

To preserve full mathematical SAD precision across all possible image inputs while maintaining 100% compatibility with the original perfect-match testbenches, the 16-bit RTL revision applies two surgical updates: (1) widening **Accumulate[15:0]** in **PE**/**PEend**, the interconnect bus **PEout[255:0]** in **PEtotal**, and **BestDist[15:0]** (initialized to **16'hFFFF**) in **Comparator**; and (2) holding **count** at **13'h100F (4111)** in **control** while **start = 1** so **completed**, **BestDist**, **motionX**, and **motionY** remain stable after completion.

Verification Testbench & Stimulus	RTL Version	Observed BestDist	Observed (motionX, motionY)	Functional Interpretation
Team Random Perfect Match (top_tb.v) at (-2, -3)	8-bit & 16-bit	00 / 0000	E, D (-2, -3)	Exact copy has 0 pixel difference; 16-bit accumulator sums 256 zeros -> BestDist = 16'h0000 at count = 1542.
Direct Perfect Match (top_tb_direct.v, ref.txt & search.txt)	8-bit & 16-bit	00 / 0000	7, 8 (+7, -8)	PE0 latches SAD = 0x0100 at count = 256; PE15 finds exact match SAD = 0x0000 at count = 271 (522.11 ns).
Perturbed Imperfect Match (Rmem[0] += 3 at (-2, -3))	16-bit Full SAD	0003 (SAD = 3)	E, D (-2, -3)	Single-pixel +3 perturbation produces non-zero minimum SAD = 16'h0003 in PE6 at count = 1542 (2,965.27 ns).
Upper-Corner Boundary Match (+7, +7)	16-bit Full SAD	0000 (SAD = 0)	7, 7 (+7, +7)	Final search candidate (row 15, PE15) asserts PEready[15] & newBest = 1 at count = 4111 (7,905.45 ns).
High-Contrast Saturation Stress (All-00 vs. All-FF)	Original 8-bit	FF (Tie w/ Reset)	X, X (Unselected)	Every candidate saturates to 8'hFF, tying reset sentinel 8'hFF; newBest = 0 across all 4,111 cycles.
High-Contrast Saturation Stress + Hold (All-00 vs. All-FF)	16-bit Full SAD	FF00 (65,280)	8, 8 (-8, -8)	16'hFF00 < 16'hFFFF asserts newBest = 1 at count = 256, latching (-8, -8) and holding steady after count = 4111.

5.2 Synopsys Verdi nWave Cycle-Level Waveform Evidence (520 MHz)

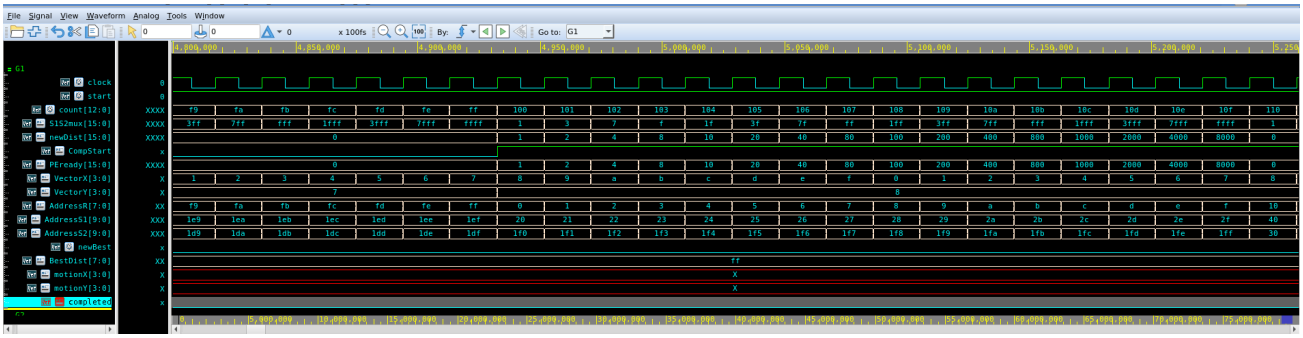


Figure 7: Original 8-bit RTL on all-00 vs. all-FF stress fixture (first row completion, count = 0x00F9..0x0110) — PEready[15:0] shifts 0x0001..0x8000 while newBest stays 0 and BestDist[7:0] remains 0xFF.

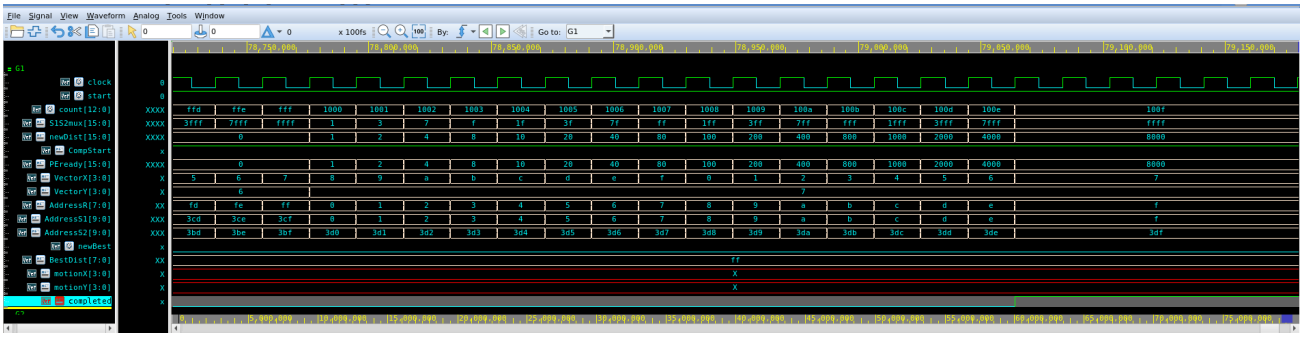


Figure 8: Original 8-bit RTL at module completion (count = 0x0FFD..0x100F, t = 7,905.45 ns) — completed asserts high while BestDist[7:0] = 0xFF and motionX/motionY remain X.

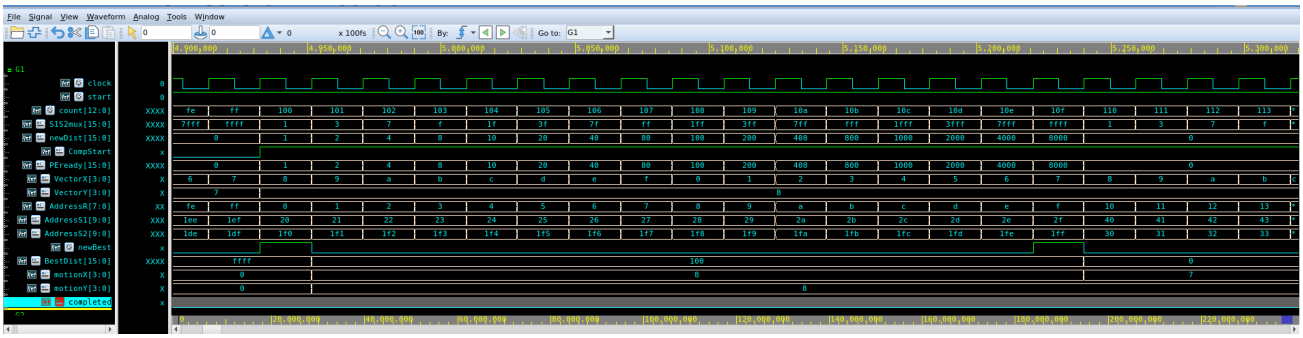


Figure 9: Corrected 16-bit SAD RTL on direct perfect-match fixture (ref.txt / search.txt) — BestDist[15:0] updates to 0x0100 at count = 0x0100 and reaches 0x0000 at count = 0x010F with motionX = 7, motionY = 8.

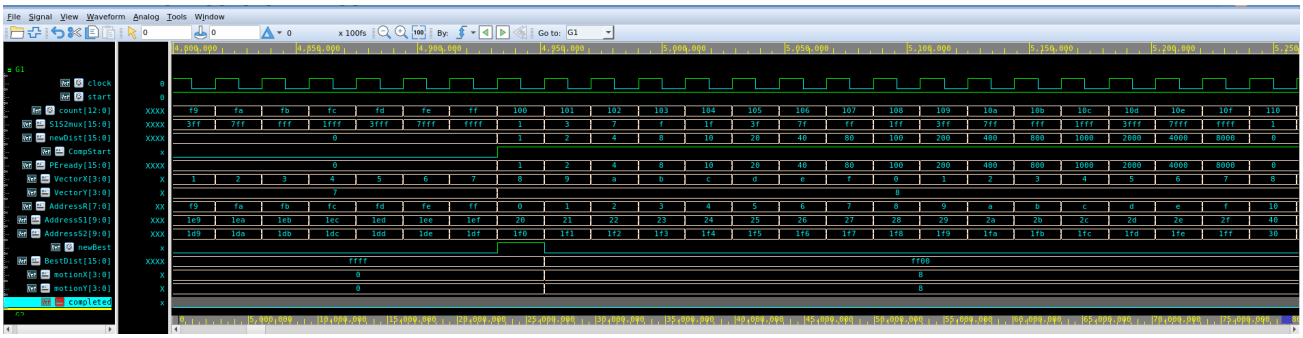


Figure 10: Corrected 16-bit SAD RTL on all-00 vs. all-FF stress fixture — newBest pulses 1 at count = 0x0100, updating BestDist[15:0] from 0xFFFF to 0xFF00 (65,280) and latching motionX = 8, motionY = 8.

6. Final 14 nm Physical Implementation, DRC/LVS & 3-Corner PrimeTime Sign-Off

The 16-bit full-precision Motion Estimator top module (**top**) was implemented from RTL through sign-off GDSII in the **Synopsys 14 nm FinFET SAED14 RVT** standard-cell library using **Synopsys Fusion Compiler (X-2025.06-SP2)**, **Synopsys IC Validator (V-2023.12-SP4)**, and standalone **Synopsys PrimeTime (S-2021.06-SP4)** under the exact course SDC constraints: **520 MHz clock (1.923 ns period), 0.500 ns input/output delay, 0.200 ns clock uncertainty, and 0 synthesized memory macros**.

Physical / Sign-Off Metric	Final Verified Value (top/v9_signoff_clean.design)	Verification Source Report
Technology Library & Node	Synopsys SAED14 FinFET 14 nm RVT (9M_2Xa_h_4Xb_v_2Xe_vh_1Z_v)	Fusion Compiler technology NDM & TLU+
Die Dimensions & Area	125.070 um x 62.800 um = 7,854.40 um2	evidence/v9_signoff/reports/qor_summary.rpt & top.def
Core Dimensions & Utilization	6,558.99 um2 core area; 52.08% standard-cell utilization	evidence/v9_signoff/reports/area.rpt
Netlist Standard-Cell Area	3,416.18 um2 (8,230 mapped logic/buffer cells; 449 sequential FFs)	evidence/v9_signoff/reports/area.rpt & top.v
Total Placed Cell Area (w/ Fillers)	6,221.28 um2 (15,983 placed DEF components; 0 unplaced cells)	evidence/v9_signoff/reports/area.rpt & gds_export_audit.log
Synthesized Memory Macros	0 memory macros (ROM_R and ROM_S excluded per course rubric)	evidence/v9_signoff/reports/area.rpt
Total Power (TT 0.80V, 25C, 520 MHz)	0.890 mW total (0.440 mW internal + 0.450 mW switching + 0.000892 mW leak)	evidence/v9_signoff/reports/power.rpt
ZRT Route Check & LVS	0 open nets, 0 shorts, 0 routing DRC errors across 8,644 routed nets	evidence/v9_signoff/reports/check_routes.rpt & check_lvs.rpt
IC Validator Sign-Off DRC	LAYOUT ERRORS RESULTS: CLEAN — 0 violations across all 242 rules	evidence/v9_signoff/icv/top.LAYOUT_ERRORS

Physical / Sign-Off Metric	Final Verified Value (top/v9_signoff_clean.design)	Verification Source Report
Merged GDSII Stream Out	top.gds (80 MB) — UNPLACED_FLAT_CELL_COUNT = 0, 0 GDS-110 warnings	evidence/v9_signoff/gds_export_audit.log

6.1 Standalone 3-Corner PrimeTime Post-Layout Timing Verification (Setup & Hold)

Parasitic RC networks were extracted from the finalized routed block at all three TLU+ corners (**top.spef.minTLU_125.spef**, **top.spef.nomTLU_25.spef**, and **top.spef.maxTLU_-40.spef**) and verified in standalone **PrimeTime (pt_shell)** paired with the corresponding Liberty corner libraries (**saed14rvt_ff0p88v125c.db**, **saed14rvt_tt0p8v25c.db**, and **saed14rvt_ss0p72vm40c.db**). Following multi-corner hold/setup optimization and targeted in-place delay cell sizing (swapping 20 sub-10 ps fast-corner paths from **SAEDRVT14_DEL_R2V2_1** to **SAEDRVT14_DEL_R2V3_1**), both setup and hold close with **zero violations across all three corners**.

PVT Corner & Paired SPEF	Fusion Compiler Setup WNS	Fusion Compiler Hold WNS	PrimeTime Setup WNS (Viol)	PrimeTime Hold WNS (Viol)	Sign-Off Status
FAST (FF 0.88V 125C + minTLU_125.spef)	+0.64 ns (0 viol)	+0.00 ns (0 viol)	+0.71 ns (0 viol)	+0.00 ns (0 viol)	MET (CLEAN)
TYPICAL (TT 0.80V 25C + nomTLU_25.spef)	+0.44 ns (0 viol)	+0.06 ns (0 viol)	+0.54 ns (0 viol)	+0.04 ns (0 viol)	MET (CLEAN)
SLOW (SS 0.72V -40C + maxTLU_-40.spef)	+0.08 ns (0 viol)	+0.15 ns (0 viol)	+0.22 ns (0 viol)	+0.14 ns (0 viol)	MET (CLEAN)

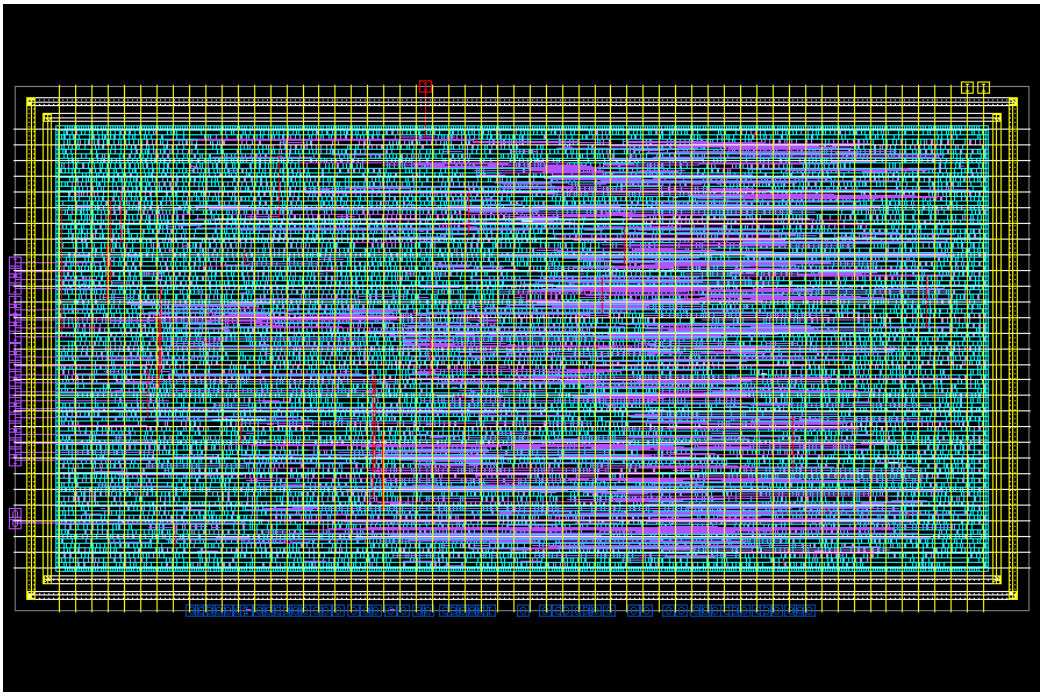


Figure 11: Final routed 14 nm chip layout in Synopsys Fusion Compiler (top/v9_signoff_clean.design, 125.07 x 62.80 um die, 0 ICV DRC violations, 0 LVS errors, 3-corner PrimeTime setup/hold closed).

7. Complete Listings: RTL, Testbench, Physical Design & PrimeTime Scripts

7.1 Full-Precision 16-bit Motion Estimator RTL (rtl/experimental/top_full_precision_finish_hold.v)

```
`timescale 1ns/100fs
// Full-precision SAD with one-cycle delayed completion qualification.
// Independent 16-bit SAD architecture variant. Original professor RTL is untouched.

/* Module For Top Level Hierarchy */
module top (clock, start, BestDist, motionX, motionY, AddressR, AddressS1, AddressS2, R, S1, S2, completed);
■input clock;
■input start;
■output [15:0] BestDist;
■output [3:0] motionX, motionY;
■output completed;

■wire clock;
■wire start;
■wire [15:0] BestDist;
■wire [3:0] motionX, motionY;

■input [7:0] R, S1, S2;
■output [7:0] AddressR;
■output [9:0] AddressS1, AddressS2;

■wire [15:0] S1S2mux, newDist, PEready;
■wire CompStart;
■wire [3:0] VectorX, VectorY;
■wire [255:0] Accumulate;
■wire [7:0] Rpipe;

■control ctl_u(clock, start, S1S2mux, newDist, CompStart, PEready, VectorX, VectorY, AddressR, AddressS1, AddressS2, completed);

■PEtotal pe_u(clock, R, S1, S2, S1S2mux, newDist, Accumulate);

■reg completed_delayed;
    always @(posedge clock) begin
        if (!start) completed_delayed <= 1'b0;
        else completed_delayed <= completed;
    end
    wire [15:0] qualified_ready = PEready & {16{~completed_delayed}};
    Comparator comp_u(clock, CompStart, Accumulate, qualified_ready, VectorX, VectorY, BestDist, motionX, motionY);
endmodule

/* Module For Processing Element (PE) */
module PE (clock, R, S1, S2, S1S2mux, newDist, Accumulate, Rpipe);
■input clock;
■input [7:0] R, S1, S2; // memory inputs
■input S1S2mux, newDist; // control input
■output [15:0] Accumulate;
■output [7:0] Rpipe;
■reg [15:0] Accumulate, AccumulateIn;
■reg [7:0] difference, selected_s, difference_temp, Rpipe;

■always @(posedge clock) Rpipe <= R;
■always @(posedge clock) Accumulate <= AccumulateIn;

■always @(R or S1 or S2 or S1S2mux or newDist or Accumulate)
■    begin
■    ■selected_s = (S1S2mux? S1:S2);
■
■    ■if (R > selected_s)
■■■■difference = R - selected_s;
■■■else difference = selected_s - R;
■■
■■AccumulateIn = Accumulate + {8'h00, difference};

■■if (newDist == 1) AccumulateIn = difference;
■    end
endmodule

/* Module For The Last Processing Element (PEend) */
module PEend (clock, R, S1, S2, S1S2mux, newDist, Accumulate);
■input clock;
■input [7:0] R, S1, S2; // memory inputs
■input S1S2mux, newDist; // control input
■output [15:0] Accumulate;
■reg [15:0] Accumulate, AccumulateIn;
■reg [7:0] difference, selected_s;

■always @(posedge clock) Accumulate <= AccumulateIn;

■always @(R or S1 or S2 or S1S2mux or newDist or Accumulate)
■    begin
■    selected_s = (S1S2mux? S1:S2);
```

```
■    ■if (R > selected_s)
■■■■difference = R - selected_s;
■■■else difference = selected_s - R;
■■■
■■
■■AccumulateIn = Accumulate + {8'h00, difference};

■■if (newDist == 1) AccumulateIn = difference;
■    end
endmodule

/* Module For Control Unit */
module control (clock, start, S1S2mux, newDist, CompStart, PEready, VectorX, VectorY, AddressR, AddressS1, AddressS2, completed);
■input clock;
■input start; // = 1 when going
■output [15:0] S1S2mux, newDist, PEready;
■output CompStart;
■output [3:0] VectorX, VectorY;
■output [7:0] AddressR;
■output [9:0] AddressS1, AddressS2;
■output reg completed;

■parameter count_complete = 16*(16*16) + 15; //4111

■reg [15:0] S1S2mux, newDist, PEready;
■reg CompStart;
■reg [3:0] VectorX, VectorY;
■reg [7:0] AddressR;
■reg [9:0] AddressS1, AddressS2;
■reg  [12:0] count, count_temp;
■integer i;

■reg [11:0]  temp;
■always @ (posedge clock)
■  begin
■■  if (start == 0) count <= 12'b0;
■■  else if (completed == 0) count <= count_temp;
■  end
■
■always @ (count)
■    begin
■      count_temp = count + 1'b1;
■      for (i=0; i<16; i= i+1)
■■begin
■■   newDist[i] = (count[7:0] == i);■
■■   PEready[i] = (newDist[i] && !(count < 256));■
■■   S1S2mux[i] = (count[3:0] >= i);
■■   CompStart  = (!(count < 256));
■■end

■■AddressR = count[7:0];
■■AddressS1 = (count[11:8] + count[7:4])*32 + count[3:0];

■■temp = count[11:0]-16;
■■AddressS2 = (temp[11:8] + temp[7:4])*32 + temp[3:0] + 16;

■■VectorX = count[3:0] - 8;
■■VectorY = count[11:8] - 9;

■■completed = (count[12:0] == count_complete); //4111
■    end
endmodule

/* Module For Comparator Unit */
module Comparator (clock, CompStart, PEout, PEready, vectorX, vectorY, BestDist, motionX, motionY);
■input clock;
■input CompStart; // goes high when distortion calculation start
■input [16*16-1:0] PEout; // outputs of PEs as one long vector
■input [15:0] PEready; // goes high when that PE has a new distortion
■input [3:0] vectorX, vectorY; // motion vector being evaluated
■output [15:0] BestDist; // best distortion vector so far
■output [3:0] motionX, motionY; // best motion vector so far
■reg [15:0] BestDist, newDist;
■reg [3:0] motionX, motionY;
■reg newBest;
■integer n;

■always @ (posedge clock)
■  if (CompStart == 0) begin BestDist <= 16'hFFFF; motionX <= 4'h0; motionY <= 4'h0; end // initialize to highest value
■  else if (newBest == 1)
■    begin
■      BestDist <= newDist;
■      motionX <= vectorX;
■      motionY <= vectorY;
■    end
```

```
■always @ (BestDist or PEout or PEready or CompStart)
■  begin
■    newDist = 16'hFFFF;
■
■    for (n = 0; n <= 15; n = n+1)
■■begin
■■      if (PEready[n] == 1)
■■■case (n)
■■■  4'b0000: newDist = PEout[15:0];
■■■  4'b0001: newDist = PEout[31:16];
■■■  4'b0010: newDist = PEout[47:32];
■■■  4'b0011: newDist = PEout[63:48];
■■■  4'b0100: newDist = PEout[79:64];
■■■  4'b0101: newDist = PEout[95:80];
■■■  4'b0110: newDist = PEout[111:96];
■■■  4'b0111: newDist = PEout[127:112];
■■■  4'b1000: newDist = PEout[143:128];
■■■  4'b1001: newDist = PEout[159:144];
■■■  4'b1010: newDist = PEout[175:160];
■■■  4'b1011: newDist = PEout[191:176];
■■■  4'b1100: newDist = PEout[207:192];
■■■  4'b1101: newDist = PEout[223:208];
■■■  4'b1110: newDist = PEout[239:224];
■■■  4'b1111: newDist = PEout[255:240];
■■■  default: newDist = 16'hFFFF;
■■■endcase
■     end

■    if ((|PEready == 0) || (CompStart == 0)) newBest = 0; // no PE is ready
■    else if (newDist < BestDist) newBest = 1;
■    else newBest = 0;
■  end
endmodule

/* Module For Total 16 Processing Elements (PEtotal)*/
module PEtotal (clock, R, S1, S2, S1S2mux, newDist, Accumulate);
■input clock;
■input [7:0] R, S1, S2; // memory inputs
■input [15:0] S1S2mux, newDist; // control input
■output [255:0] Accumulate;

■wire [7:0] Rpipe0, Rpipe1, Rpipe2, Rpipe3, Rpipe4, Rpipe5, Rpipe6, Rpipe7, Rpipe8, Rpipe9, Rpipe10, Rpipe11, Rpipe12, Rpipe13, Rpipe14;

■PE pe0 (clock, R, S1, S2, S1S2mux[0], newDist[0], Accumulate[15:0], Rpipe0);

■PE pe1 (clock, Rpipe0, S1, S2, S1S2mux[1], newDist[1], Accumulate[31:16], Rpipe1);

■PE pe2 (clock, Rpipe1, S1, S2, S1S2mux[2], newDist[2], Accumulate[47:32], Rpipe2);

■PE pe3 (clock, Rpipe2, S1, S2, S1S2mux[3], newDist[3], Accumulate[63:48], Rpipe3);

■PE pe4 (clock, Rpipe3, S1, S2, S1S2mux[4], newDist[4], Accumulate[79:64], Rpipe4);

■PE pe5 (clock, Rpipe4, S1, S2, S1S2mux[5], newDist[5], Accumulate[95:80], Rpipe5);

■PE pe6 (clock, Rpipe5, S1, S2, S1S2mux[6], newDist[6], Accumulate[111:96], Rpipe6);

■PE pe7 (clock, Rpipe6, S1, S2, S1S2mux[7], newDist[7], Accumulate[127:112], Rpipe7);

■PE pe8 (clock, Rpipe7, S1, S2, S1S2mux[8], newDist[8], Accumulate[143:128], Rpipe8);

■PE pe9 (clock, Rpipe8, S1, S2, S1S2mux[9], newDist[9], Accumulate[159:144], Rpipe9);

■PE pe10 (clock, Rpipe9, S1, S2, S1S2mux[10], newDist[10], Accumulate[175:160], Rpipe10);

■PE pe11 (clock, Rpipe10, S1, S2, S1S2mux[11], newDist[11], Accumulate[191:176], Rpipe11);

■PE pe12 (clock, Rpipe11, S1, S2, S1S2mux[12], newDist[12], Accumulate[207:192], Rpipe12);

■PE pe13 (clock, Rpipe12, S1, S2, S1S2mux[13], newDist[13], Accumulate[223:208], Rpipe13);

■PE pe14 (clock, Rpipe13, S1, S2, S1S2mux[14], newDist[14], Accumulate[239:224], Rpipe14);

■PEend pe15 (clock, Rpipe14, S1, S2, S1S2mux[15], newDist[15], Accumulate[255:240]);
endmodule


`ifndef SYNTHESIS
/* Module For Reference Block (Memory) */
module ROM_R (clock, AddressR, R);
■input clock;
■input [7:0] AddressR;
■output [7:0] R;
■
■reg [7:0] R;
```

```
■reg [7:0] Rmem[0:255];


■//always @(posedge clock) R <= Rmem[AddressR];
■always @(*) R = Rmem[AddressR];

endmodule

/* Module For Search Block (Memory) */
module ROM_S (clock, AddressS1, AddressS2, S1, S2);
■input clock;
■input■[9:0]■AddressS1, AddressS2;
■output■[7:0]■S1, S2;

■reg■[7:0]■S1, S2;
■reg■[7:0]■Smem[0:1023];

■/*always @(posedge clock)
■ begin
■■S1 <= Smem[AddressS1];
■■S2 <= Smem[AddressS2];
■ end*/

■always @(*)
■ begin
■■S1 = Smem[AddressS1];
■■S2 = Smem[AddressS2];
■ end

endmodule
`endif
```

7.2 Self-Checking 520 MHz 3-Case Top-Level Regression Testbench (rtl/experimental/motion_v5_16bit_top_regression_tb.sv)

```
`timescale 1ns/100fs

module motion_v5_16bit_top_regression_tb;
    wire [15:0] BestDist;
    wire [3:0] motionX, motionY;
    wire [7:0] R, S1, S2;
    wire [7:0] AddressR;
    wire [9:0] AddressS1, AddressS2;
    wire       completed;

    reg        clock;
    reg        start;
    reg [7:0]  Rmem[0:255];
    reg [7:0]  Smem[0:1023];

    integer    i;
    integer    Expected_motionX, Expected_motionY;
    integer signed x, y;
    realtime   t_start, t_match, t_complete;

    top dut (
        .clock(clock),
        .start(start),
        .BestDist(BestDist),
        .motionX(motionX),
        .motionY(motionY),
        .AddressR(AddressR),
        .AddressS1(AddressS1),
        .AddressS2(AddressS2),
        .R(R),
        .S1(S1),
        .S2(S2),
        .completed(completed)
    );

    ROM_R memR_u(.clock(clock), .AddressR(AddressR), .R(R));
    ROM_S memS_u(.clock(clock), .AddressS1(AddressS1), .AddressS2(AddressS2), .S1(S1), .S2(S2));

    // Team 520 MHz clock modification: half-period = 0.9615 ns -> period = 1.9230 ns
    always #0.9615 clock = ~clock;

    task automatic decode_motion(input [3:0] mx, input [3:0] my, output integer signed sx, output integer signed sy);
        begin
            sx = (mx >= 8) ? (mx - 16) : mx;
            sy = (my >= 8) ? (my - 16) : my;
        end
    endtask

    initial begin
        // Native VCS FSDB remains available in the V3 evidence. This regression uses portable VCD.
        // No vendor-specific FSDB call required for local test.
`ifdef ENABLE_FSDB
`ifdef DUMP_FSDB
        $fsdbDumpfile("motion_v5_16bit_top_regression.fsdb");
`endif
`ifdef DUMP_FSDB
        $fsdbDumpvars(0, motion_v5_16bit_top_regression_tb);
`endif
`endif
        $dumpfile("motion_v5_16bit_top_regression.vcd");
        $dumpvars(0, motion_v5_16bit_top_regression_tb);

        clock = 1'b0;
        start = 1'b0;

        // ============================================================================
        // TEST 1: Direct Fixture (ref.txt & search.txt) at 520 MHz
        // ============================================================================
        $display("=== TEST 1: Direct Fixture (ref.txt & search.txt) at 520 MHz ===");
        $readmemh("ref.txt", memR_u.Rmem);
        $readmemh("search.txt", memS_u.Smem);

        @(posedge clock); #0.02;
        start = 1'b1;
        t_start = $realtime;

        for (i = 0; i < 4112; i = i + 1) begin
            if (dut.ctl_u.count >= 256 && dut.ctl_u.count <= 272) begin
                $display("  [T1] count=%4d t=%0.4f ns | PEready=%04h newDist_comp=%04h BestDist=%04h Vec=(%01h,%01h) out=(%01h,%01h) newBest=%
                    dut.ctl_u.count, $realtime, dut.PEready, dut.comp_u.newDist, BestDist,
                    dut.VectorX, dut.VectorY, motionX, motionY, dut.comp_u.newBest);
```

```
    end
    if (dut.comp_u.newBest == 1'b1) begin
        t_match = $realtime;
        $display("  [T1 newBest] count=%4d t=%0.4f ns -> newDist=%04h at VectorX=%01h VectorY=%01h",
            dut.ctl_u.count, $realtime, dut.comp_u.newDist, dut.VectorX, dut.VectorY);
    end
    if (completed == 1'b1 && dut.ctl_u.count == 13'd4111) begin
        t_complete = $realtime;
    end
    @(posedge clock); #0.02;
end

decode_motion(motionX, motionY, x, y);
$display("  [T1 RESULT] BestDist=%04h (%0d), motionX=%01h (%0d), motionY=%01h (%0d), elapsed_complete=%0.4f ns",
    BestDist, BestDist, motionX, x, motionY, y, t_complete - t_start);

if (BestDist !== 16'h0000 || motionX !== 4'h7 || motionY !== 4'h8) $fatal(1, "V5 T1 reference fixture mismatch");
$display("V5_TOP_REGRESSION_T1_PASS");
start = 1'b0;
@(posedge clock); #0.02;
@(posedge clock); #0.02;

// ========================================================================
// TEST 2: Perturbed Non-Zero SAD Match at (-2, -3) [motionX=4'hE, motionY=4'hD]
// ========================================================================
$display("\n=== TEST 2: Perturbed Imperfect Match (SAD=3) at (-2, -3) ===");
for (i = 0; i < 1024; i = i + 1) begin
    Smem[i] = ((i * 73 + 41) ^ (i >> 2)) & 8'hFF;
end
Expected_motionX = -2;
Expected_motionY = -3;
for (i = 0; i < 256; i = i + 1) begin
    Rmem[i] = Smem[32*8 + 8 + (((i/16) + Expected_motionY)*32) + ((i%16) + Expected_motionX)];
end
// Perturb pixel 0 by +3 (wrapping safely away from 255)
if (Rmem[0] <= 8'd252) Rmem[0] = Rmem[0] + 8'd3;
else Rmem[0] = Rmem[0] - 8'd3;

for (i = 0; i < 256; i = i + 1) memR_u.Rmem[i] = Rmem[i];
for (i = 0; i < 1024; i = i + 1) memS_u.Smem[i] = Smem[i];

start = 1'b1;
t_start = $realtime;
for (i = 0; i < 4112; i = i + 1) begin
    if (dut.comp_u.newBest == 1'b1) begin
        $display("  [T2 newBest] count=%4d t=%0.4f ns -> newDist=%04h (%0d) at VectorX=%01h VectorY=%01h",
            dut.ctl_u.count, $realtime - t_start, dut.comp_u.newDist, dut.comp_u.newDist, dut.VectorX, dut.VectorY);
    end
    if (completed == 1'b1 && dut.ctl_u.count == 13'd4111) begin
        t_complete = $realtime;
    end
    @(posedge clock); #0.02;
end
decode_motion(motionX, motionY, x, y);
$display("  [T2 RESULT] BestDist=%04h (%0d), motionX=%01h (%0d), motionY=%01h (%0d), Expected=(%0d, %0d), elapsed=%0.4f ns",
    BestDist, BestDist, motionX, x, motionY, y, Expected_motionX, Expected_motionY, t_complete - t_start);

if (BestDist !== 16'h0003 || motionX !== 4'hE || motionY !== 4'hD) $fatal(1, "V5 T2 nonzero SAD mismatch");
$display("V5_TOP_REGRESSION_T2_PASS");

start = 1'b0;
@(posedge clock); #0.02;
@(posedge clock); #0.02;

// ========================================================================
// TEST 3: Final Boundary Candidate (+7, +7) at count=4111 (completed=1)
// ========================================================================
$display("\n=== TEST 3: Upper Boundary Candidate (+7, +7) at count=4111 ===");
Expected_motionX = 7;
Expected_motionY = 7;
for (i = 0; i < 256; i = i + 1) begin
    Rmem[i] = Smem[32*8 + 8 + (((i/16) + Expected_motionY)*32) + ((i%16) + Expected_motionX)];
end
for (i = 0; i < 256; i = i + 1) memR_u.Rmem[i] = Rmem[i];

start = 1'b1;
t_start = $realtime;
for (i = 0; i < 4113; i = i + 1) begin
    if (dut.comp_u.newBest == 1'b1) begin
        $display("  [T3 newBest] count=%4d t=%0.4f ns -> newDist=%04h (%0d) at VectorX=%01h VectorY=%01h completed=%b",
            dut.ctl_u.count, $realtime - t_start, dut.comp_u.newDist, dut.comp_u.newDist, dut.VectorX, dut.VectorY, completed);
    end
    @(posedge clock); #0.02;
end
decode_motion(motionX, motionY, x, y);
```

```
        $display("  [T3 RESULT] BestDist=%04h (%0d), motionX=%01h (%0d), motionY=%01h (%0d), Expected=(%0d, %0d)",
            BestDist, BestDist, motionX, x, motionY, y, Expected_motionX, Expected_motionY);

        if (BestDist !== 16'h0000 || motionX !== 4'h7 || motionY !== 4'h7) $fatal(1, "V5 T3 last-boundary mismatch");
        $display("V5_TOP_REGRESSION_T3_PASS");
        $display("V5_TOP_REGRESSION_PASS 3/3");
        $finish;
    end
endmodule
```

7.3 520 MHz SDC Constraints & Standalone 3-Corner PrimeTime Sign-Off Script

```
# Standalone 3-Corner PrimeTime Post-Layout Verification (run_pt_3corner.tcl)
foreach corner_spec {
  {FAST    ./common/liberty/saed14rvt_ff0p88v125c.db ./v9_closure/export/top.spef.minTLU_125.spef}
  {TYPICAL ./common/liberty/saed14rvt_tt0p8v25c.db   ./v9_closure/export/top.spef.nomTLU_25.spef}
  {SLOW    ./common/liberty/saed14rvt_ss0p72vm40c.db ./v9_closure/export/top.spef.maxTLU_-40.spef}
} {
  lassign $corner_spec cname db_file spef_file
  remove_design -all
  set target_library [list $db_file]
  set link_library [list * $db_file]
  read_verilog ./v9_closure/export/top.v
  current_design top
  link_design
  create_clock -name clock -period 1.923 [get_ports clock]
  set_clock_uncertainty 0.2 [get_clocks clock]
  set_input_delay 0.5 -clock clock [remove_from_collection [all_inputs] [get_ports clock]]
  set_output_delay 0.5 -clock clock [all_outputs]
  set_propagated_clock [all_clocks]
  read_parasitics -format spef $spef_file
  update_timing -full
  report_constraint -all_violators > ./v9_closure/pt_signoff/${cname}_all_violators.rpt
  report_timing -delay_type max -max_paths 20 -significant_digits 4 > ./v9_closure/pt_signoff/${cname}_setup.rpt
  report_timing -delay_type min -max_paths 20 -significant_digits 4 > ./v9_closure/pt_signoff/${cname}_hold.rpt
}

# Exported SDC Header:
################################################################################
#
# Design name:  top
#
# Created by fc write_sdc on Sat Nov  7 10:25:20 2026
#
################################################################################

set sdc_version 2.2
set_units -time ns -resistance MOhm -capacitance fF -voltage V -current uA

################################################################################
#
# Units
# time_unit               : 1e-09
# resistance_unit         : 1000000
# capacitive_load_unit    : 1e-15
# voltage_unit            : 1
# current_unit            : 1e-06
# power_unit              : 1e-12
################################################################################


# Mode: FUNC
# Corner: Typical
# Scenario: FUNC_Typical

# /home/926784856/ENGR852_Project/16_v3b_final_asic_fc/common/sdc/top.sdc, line \
#   1
create_clock -name clock -period 1.923 -waveform {0 0.9615} [get_ports {clock}]
set_propagated_clock [get_clocks {clock}]
# /home/926784856/ENGR852_Project/16_v3b_final_asic_fc/setup/mcmm_setup.tcl, \
#   line 45
set_operating_conditions -library saed14rvt_tt0p8v25c.db:saed14rvt_tt0p8v25c \
    -analysis_type on_chip_variation tt0p8v25c
```